2.4.1.7 Connection lease time metric

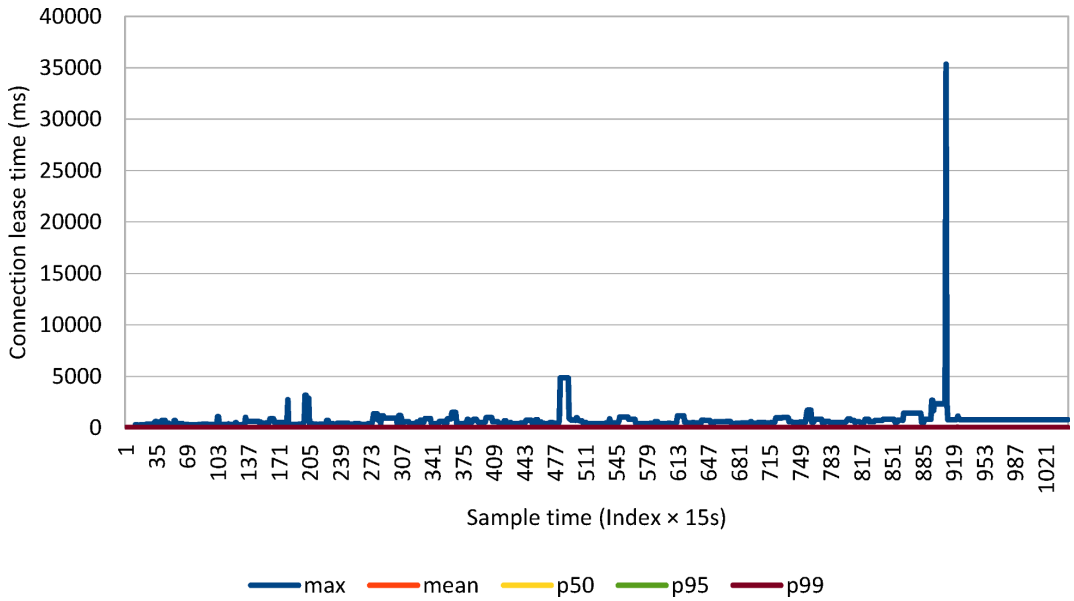

Figure 2.15: Connection lease time

The 99th percentile indicates a rather stable connection lease time throughout the whole job execution. On the other hand, the maximum graph shows a long-running transaction lasting over 35 seconds.

> Holding connections for long periods of time can increase the connection acquisition time, and fewer resources are available to other incoming clients.
>
> Most often, connections are leased for the whole duration of a database transaction. Long-running transactions might hold database locks, which, in turn, might lead to increasing the serial portion of the current execution context, therefore hindering parallelism.
>
> Long-running transactions can be addressed by properly indexing slow queries or by splitting the application-level transaction over multiple database transactions like it is the case in many ETL (Extract, Transform, and Load) systems.

3. Batch Updates

JDBC 2.0 introduced *batch updates* so that multiple DML statements can be grouped into a single database request. Sending multiple statements in a single request reduces the number of database roundtrips, therefore decreasing transaction response time. Even if the reference specification uses the term *updates*, any *insert*, *update* or *delete* statement can be batched, and JDBC supports batching for `java.sql.Statement`, `java.sql.PreparedStatement` and `java.sql.CallableStatement` too.

Not only each database driver is distinct, but even different versions of the same driver might require implementation-specific configurations.

3.1 Batching Statements

For executing static SQL statements, JDBC defines the `Statement` interface, which comes with a batching API as well. Other than for test sake, using a `Statement` for CRUD (Create, Read, Update, Delete), as in the example below, should be avoided for it's prone to SQL injection attacks.

```
statement.addBatch(
    "INSERT INTO post (title, version, id) " +
    "VALUES ('Post no. 1', 0, default)");

statement.addBatch(
    "INSERT INTO post_comment (post_id, review, version, id) " +
    "VALUES (1, 'Post comment 1.1', 0, default)");

int[] updateCounts = statement.executeBatch();
```

The numbers of database rows affected by each statement is included in the return value of the `executeBatch()` method.

Oracle

For `Statement` and `CallableStatement`, the Oracle JDBC Driver[a] does not actually support batching. For anything but `PreparedStatement`, the driver ignores batching, and each statement is executed separately.

[a] http://docs.oracle.com/cd/E11882_01/java.112/e16548/oraperf.htm#JJDBC28752

The following graph depicts how different JDBC drivers behave when varying batch size, the test measuring the time it takes to insert 1000 *post* rows with 4 *comments* each:

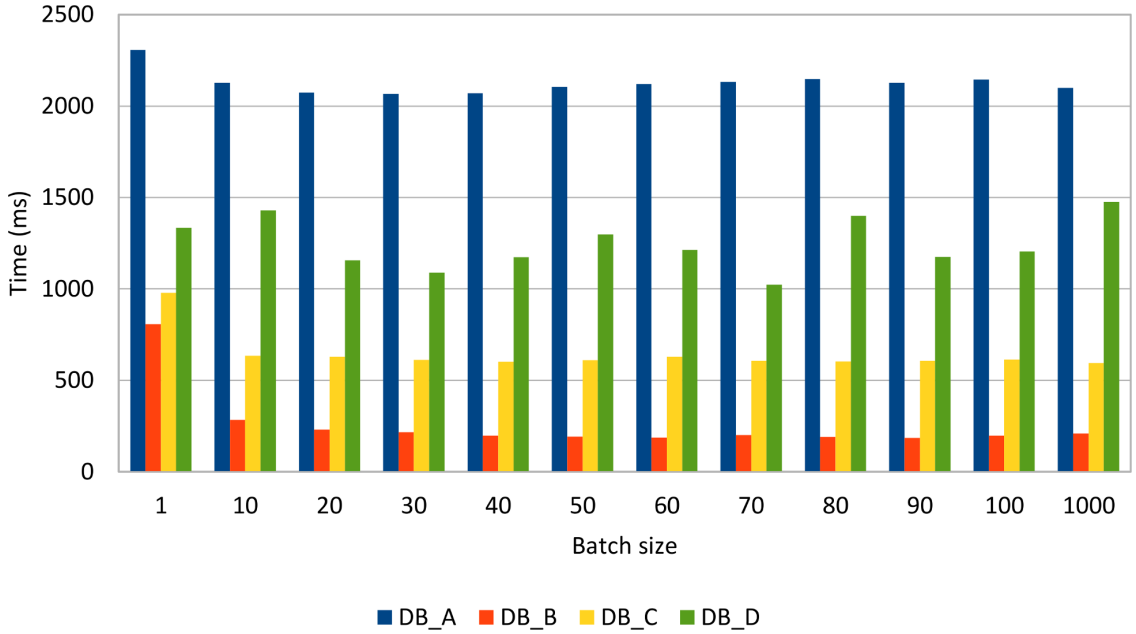

Figure 3.1: Statement batching

Reordering inserts, so that all *posts* are inserted before the *comment* rows, gives the following results:

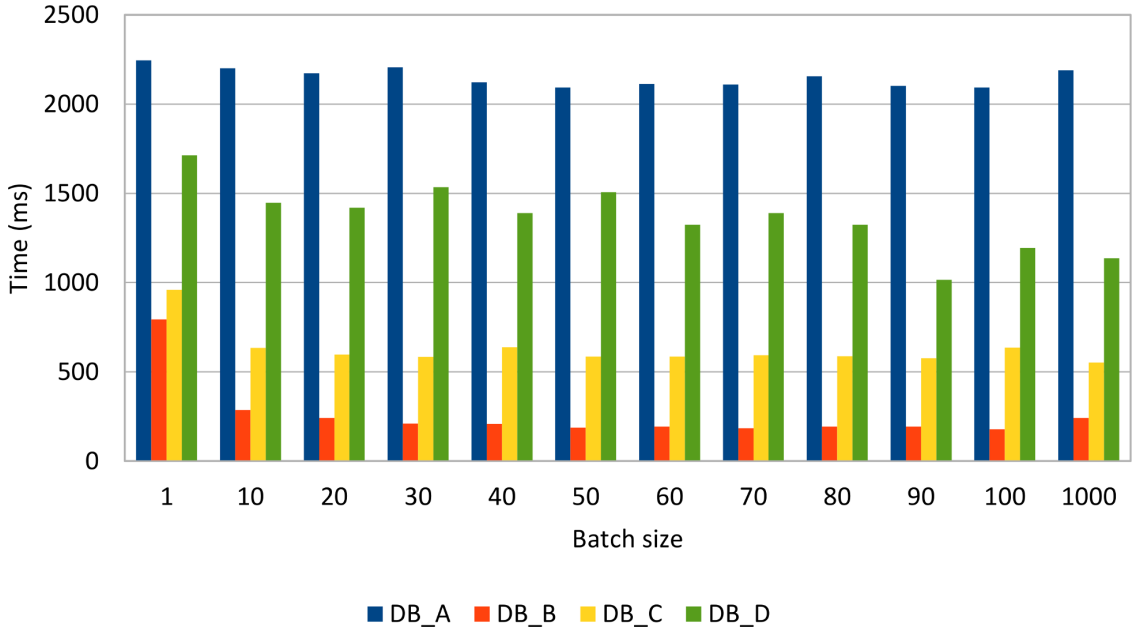

Figure 3.2: Reordered statement batching

Reordering statements does not seem to improve performance noticeably, although some drivers (e.g. MySQL) might take advantage of this optimization.

MySQL

Although it implements the JDBC specification, by default, the MySQL JDBC driver does not send the batched statements in a single request.

For this purpose, the JDBC driver defines the `rewriteBatchedStatements`[a] connection property, so that statements get rewritten into a single `String` buffer. In order to fetch the auto-generated row keys, the batch must contain insert statements only.

For `PreparedStatement`, this property rewrites the batched insert statements into a multi-value insert. Unfortunately, the driver is not able to use server-side prepared statements when enabling rewriting.

Without setting this property, the MySQL driver simply executes each DML statement separately, therefore defeating the purpose of batching.

[a] http://dev.mysql.com/doc/connector-j/en/connector-j-reference-configuration-properties.html

The following graph demonstrates how statement rewriting performs against the default behavior of the MySQL JDBC driver:

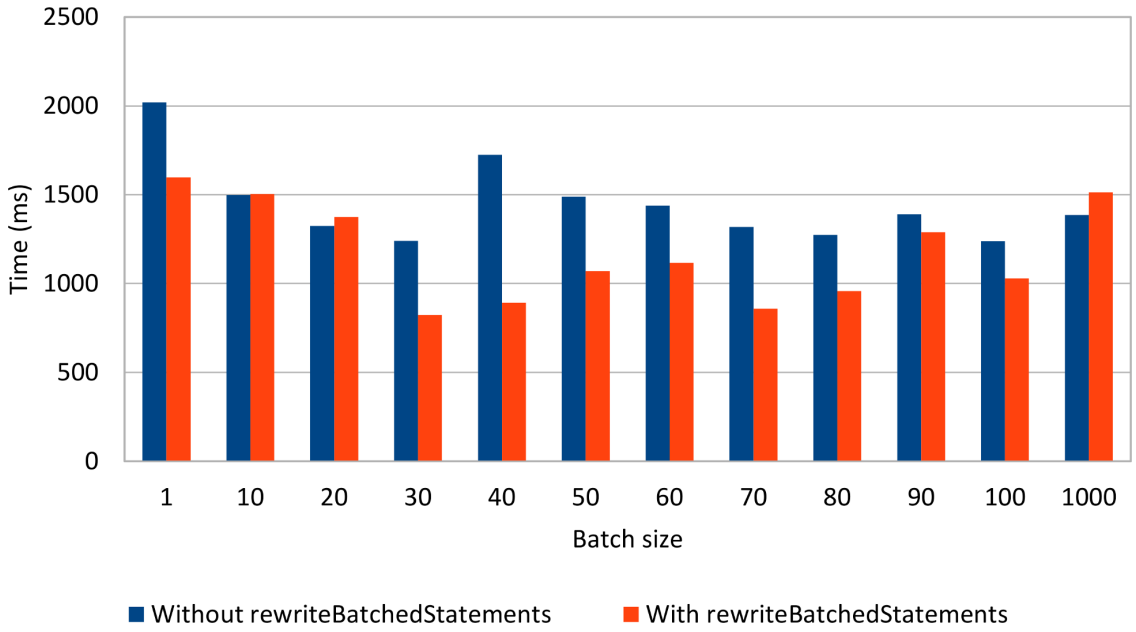

Figure 3.3: MySQL Statement batching

Rewriting non-parameterized statements seems to make a difference, as long as the batch size is not too large. In practice, it is common to use a relatively small batch size, to reduce both the client-side memory footprint and to avoid congesting the server from suddenly processing a huge batch load.

3.2 Batching PreparedStatements

For parameterized statements (a very common enterprise application requirement), the JDBC `Statement` is a poor fit because the only option for varying the executing SQL statement is through `String` manipulation. Using a `String` template or concatenating `String` tokens is risky as it makes the data access logic vulnerable to SQL injection attacks.

To address this shortcoming, JDBC offers the `PreparedStatement` interface for binding parameters in a safe manner. The driver must validate the provided parameter at runtime, therefore discarding unexpected input values.

Because a `PreparedStatement` is associated with a single DML statement, the batch update can group multiple parameter values belonging to the same prepared statement.

```
PreparedStatement postStatement = connection.prepareStatement(
    "INSERT INTO post (title, version, id) " +
    "VALUES (?, ?, ?)");

postStatement.setString(1, String.format("Post no. %1$d", 1));
postStatement.setInt(2, 0);
postStatement.setLong(3, 1);
postStatement.addBatch();

postStatement.setString(1, String.format("Post no. %1$d", 2));
postStatement.setInt(2, 0);
postStatement.setLong(3, 2);
postStatement.addBatch();

int[] updateCounts = postStatement.executeBatch();
```

SQL injection

For an enterprise application, security is a very important technical requirement. The SQL *Injection* attack exploits data access layers that do not use bind parameters. When the SQL statement is the result of `String` concatenation, an attacker could inject a malicious SQL routine that is sent to the database along the current executing statement.

SQL injection is usually done by ending the current statement with the `;` character and continuing it with a rogue SQL command, like modifying the database structure (deleting a table or modifying authorization rights) or even extracting sensitive information.